are communicated during the handshake and some clients may fail if there are both TLS and DTLS versions in the list.

- Just like the `DtlsDatagramChannel` you used before, Bouncy Castle requires the UDP socket to be connected before the DTLS handshake occurs. Because the server doesn't know where the client is located, you can wait until a packet is received from any client and then call `connect()` with the socket address of the client.
- Create a `DTLSServerProtocol` and `UDPTransport` objects, and then call the accept method on the protocol object to perform the DTLS handshake. This returns a `DTLSTransport` object that you can then use to send and receive encrypted and authenticated packets with the client.

TIP Although the Bouncy Castle API is straightforward when using PSKs, I find it cumbersome and hard to debug if you want to use certificate authentication and I prefer the `SSLEngine` API.

Listing 12.9 DTLS PSK server

```
public static void main(String[] args) throws Exception {
    var psk = loadPsk(args[0].toCharArray());    #A
    var crypto = new BcTlsCrypto(new SecureRandom());
    var server = new PSKTlsServer(crypto,
getIdentityManager(psk)) {    #B
        @Override    #B
        protected ProtocolVersion[] getSupportedVersions()
{    #B
            return ProtocolVersion.DTLSv12.only();    #B
        }    #B
    };    #B
```